

Does Structured Trajectory Memory Improve Agent Retrieval? Evaluating ADP as a Representation for Experience-Augmented Agents

Anonymous Authors
Paper under double-blind review

April 14, 2026

Abstract

Agent Data Protocol (ADP) was originally introduced as a standardized and expressive schema for representing heterogeneous agent trajectories, with demonstrated value for offline agent training and fine-tuning. In this paper, we study a different question: whether ADP is also a better *retrieval representation* for experience-augmented agents. We consider a retrieval setting in which an agent can query a memory store of prior trajectories during live execution, retrieve candidate past experiences, and optionally inject them back into context. Our central hypothesis is that ADP preserves procedural structure—tool usage, action types, environment observations, and interaction flow—that is lost in weaker memory representations such as flattened text logs or short natural-language summaries. To test this, we build a reproducible evaluation pipeline that converts agent sessions into ADP, indexes them as memory items, and compares ADP-based retrieval against unstructured baselines across tool-using agent tasks. We evaluate not only task success, but also retrieval usefulness, injection harm, search cost, and robustness under noisy or stale memory. Our results aim to answer three questions: (1) whether ADP improves procedural memory retrieval, (2) which structural components of ADP matter most, and (3) when retrieval is beneficial versus harmful in live agent execution. Taken together, the paper positions ADP not only as a training interlingua, but also as a candidate memory substrate for retrieval-augmented agents.

1 Introduction

Large language model agents increasingly operate through multi-step interactions with tools, environments, and users. In such settings, purely parametric memory is often insufficient: agents may repeatedly encounter similar subproblems, failure modes, and tool-use patterns across tasks. A natural idea is to

augment a live agent with a memory store of prior trajectories and retrieve relevant experience when needed.

However, the effectiveness of such retrieval depends not only on the retrieval algorithm but also on the *representation* of the stored experience itself. Most existing memory implementations flatten trajectories into plain text, short summaries, or ad hoc log formats. These formats are convenient, but they often discard procedural structure: tool invocations, typed actions, environment feedback, and intermediate failure signals may be lost or blurred into unstructured text. This makes it unclear whether current retrieval-based agent memory is underperforming because of weak retrieval policies, weak storage representations, or both.

This paper investigates whether ADP, originally proposed as a standardized representation for offline agent training data, is also a better representation for retrieval-based agent memory. The key intuition is simple: if memory items preserve the procedural structure of past experience, then retrieval and post-retrieval decision-making should be able to exploit more than superficial semantic overlap.

We study this question in a controlled setting built around a memory system called RECALL. In RECALL, a live agent can query a trajectory store, retrieve candidate prior sessions, and decide whether to inject nothing, a compact summary, or a fuller memory artifact into context. A key design constraint is that some signals needed for relevance estimation, especially semantic similarity to stored trajectories, are only available after retrieval has been executed. This motivates a hierarchical retrieval-control view in which local search decisions and post-search injection decisions are separated.

The paper makes the following contributions:

1. We formulate *trajectory representation for agent memory* as a scientific evaluation problem, rather than a purely systems problem.
2. We build a reproducible evaluation pipeline that converts agent sessions into ADP memory items, stores them in a searchable memory substrate, and supports controlled comparison against weaker representations.
3. We compare ADP against unstructured baselines such as flattened logs, message-only traces, and natural-language summaries.
4. We analyze not only whether retrieval helps, but also when it hurts, which parts of ADP matter, and how retrieval-control interacts with memory quality.

Our goal is not to claim that ADP universally solves agent memory. Rather, the goal is to determine whether structured, typed trajectories provide a measurable advantage as memory units for retrieval-augmented agent execution.

2 Research Questions and Hypotheses

We center the paper around the following research questions.

RQ1. Does ADP-based memory retrieval improve downstream agent performance compared with weaker trajectory representations such as flattened text logs, message-only traces, or summary memory?

RQ2. Which components of ADP matter most for retrieval effectiveness: typed actions, tool arguments, observations, metadata, or their interaction?

RQ3. Does ADP help more on procedural and tool-heavy tasks than on primarily conversational tasks?

RQ4. When is retrieval beneficial, and when is it harmful, in the presence of noisy, stale, or weakly matched memory?

We test the following hypotheses.

H1. ADP outperforms unstructured memory representations on tasks where procedural reuse matters.

H2. The gain from ADP comes primarily from preserving typed tool/action structure and environment observations, not merely from storing more tokens.

H3. Naive memory usage is often harmful or wasteful; the benefit of structured memory is largest when combined with controlled retrieval and selective injection.

3 Background and Related Work

3.1 ADP as a trajectory standard

Briefly summarize the original ADP paper here:

- motivation: heterogeneous agent datasets are difficult to combine;
- design principles: simplicity, standardization, expressiveness;
- schema: trajectory id, content sequence, flexible details metadata;
- action types: API actions, code actions, message actions;
- observation types: text observations, web observations;
- original demonstrated use case: offline fine-tuning / training.

Important framing sentence: The original ADP work studies *training-time* utility; our paper studies *retrieval-time* utility.

3.2 Memory and retrieval in LLM agents

Review agent memory systems, trajectory retrieval, episodic memory, procedural memory, experience replay for agents, and agentic RAG. Emphasize the gap: most prior work studies retrieval policy or memory storage, but not the effect of the *trajectory representation itself*.

3.3 Evaluation as a scientific object

Because this paper targets the E&D track, make the evaluation contribution explicit. Argue that representation choice can change what retrieval claims are even measurable, and therefore belongs in the science of evaluation.

3.4 Positioning of this paper

State clearly: This paper is *not* proposing ADP itself. It asks whether ADP, originally justified for training, also serves as a superior memory substrate for retrieval-based agents.

4 Problem Setup

We consider a live agent interacting with a user and an external environment over discrete decision steps $t = 1, 2, \dots$. At each step, the agent may emit a message, call a tool, receive an observation, or replan. Alongside this live execution, the system maintains a memory store $\mathcal{M}$ of prior trajectories.

A memory item $m \in \mathcal{M}$ is derived from a past session and stored under one of several representations:

$$r \in \mathcal{R} = \{\text{flat-text}, \text{message-only}, \text{summary}, \text{ADP}, \text{ADP-lite}\}.$$

At selected steps, the live session may issue a retrieval query q_t , obtain a candidate set

$$C_t = \{c_1, \dots, c_k\},$$

and optionally inject a memory-derived artifact into the agent context.

We separate the problem into two layers:

1. **Representation quality:** does the chosen stored format make relevant experience easier to retrieve and use?
2. **Retrieval-control quality:** given a representation, when should the system search, and when should it inject?

This paper focuses primarily on the first question and treats retrieval-control as either (a) a controlled baseline factor, or (b) a secondary analysis dimension.

4.1 Evaluation targets

We evaluate memory systems along five axes:

1. task success,
2. retrieval usefulness,
3. injection harm,
4. search/context cost,
5. robustness to memory noise and staleness.

4.2 Why representation matters

Add a short intuitive example here: two past sessions may share procedural structure while differing lexically. A plain-text or summary representation may miss this similarity, while ADP may preserve it through typed tool calls, action ordering, and environment feedback.

5 System Overview: Recall

5.1 Current implementation

Describe only what is actually implemented. This section should be factual and conservative.

Session normalization. We convert OPENHANDS session logs into ADP-shaped trajectories. The converter maps user messages, agent messages, terminal actions, file editor actions, tool-related events, and environment observations into typed ADP items.

Embedding and storage. Each converted trajectory is embedded and stored as a single indexed memory item together with metadata and the full ADP content.

Memory backend. The current prototype uses a vector store for approximate similarity search over prior trajectories.

Important scope boundary. The current repository implements offline trajectory conversion and ingestion. Any online retrieval/injection loop used in experiments must be described separately and clearly marked as part of the experimental harness.

5.2 Memory representations compared

Define each representation formally.

Flat-text memory. Flatten the entire trajectory into a single text string.

Message-only memory. Retain only user and agent natural-language messages; discard tool/action structure.

Summary memory. Compress the trajectory into a short natural-language summary of fixed budget.

ADP memory. Store the typed action-observation trajectory and metadata as structured memory.

ADP-lite memory. Ablated structured variant that removes selected components, e.g. tool kwargs or observations.

5.3 Optional retrieval-control layer

If you include the trigger, keep it secondary. Do *not* make the trigger the main contribution unless experiments justify it.

A minimal paragraph could say:

For part of our analysis, we consider a hierarchical retrieval-control policy in which the live wrapper first decides whether to search, after which post-retrieval signals determine whether to inject no memory, a compact summary, or a fuller artifact. This decomposition reflects the fact that some relevance signals are only available after candidate retrieval.

6 Experimental Design

6.1 Task families

Describe the task suite. A strong structure is:

- **Procedural/tool-heavy tasks:** coding, debugging, file editing, web workflows.
- **Mixed tasks:** tasks with both language interaction and tool use.
- **Low-procedural/conversational controls:** tasks where memory structure should matter less.

6.2 Memory corpus construction

Explain where stored trajectories come from. Possible sources:

- historical OPENHANDS sessions,
- replayed benchmark trajectories,

- held-out prior episodes from the same domain,
- synthetic perturbations for noise/staleness analysis.

Train/validation/test split for memory. State how memory items and evaluation tasks are separated. This is crucial. Avoid leakage between the memory bank and target episodes.

6.3 Compared methods

Your core table should compare:

M1: No memory.

M2: Flat-text memory.

M3: Message-only memory.

M4: Summary memory.

M5: ADP-lite memory.

M6: Full ADP memory.

Then optionally compare retrieval-control variants:

C1: Always retrieve.

C2: Fixed interval retrieve.

C3: Heuristic retrieve.

C4: Hierarchical trigger.

6.4 Metrics

Define each metric carefully.

Task success. Binary or graded completion metric appropriate to the task.

Retrieval precision@k or candidate usefulness. Whether retrieved items are actually relevant for the live subproblem.

Injection usefulness. Whether the injected artifact measurably improves downstream behavior.

Injection harm rate. Fraction of injection events that degrade task progress, waste tokens, or mislead the agent.

Search cost and context cost. Latency, number of retrieval calls, tokens injected, and extra context budget consumed.

Table 1: Main comparison across memory representations. Higher is better except harm and cost.

Method	Success $\uparrow$	Usefulness $\uparrow$	Harm $\downarrow$	Searches $\downarrow$	Cost $\downarrow$
No memory	–	–	–	–	–
Flat-text	–	–	–	–	–
Message-only	–	–	–	–	–
Summary	–	–	–	–	–
ADP-lite	–	–	–	–	–
ADP	–	–	–	–	–

Robustness metrics. Performance under noisy, stale, contradictory, or weakly matching memories.

6.5 Statistical testing

State confidence intervals, number of seeds / runs, and statistical tests. Examples: paired bootstrap, paired permutation test, stratified confidence intervals by task family.

7 Main Results

7.1 Does ADP improve retrieval-augmented performance?

Present the main results table here.

What to write here. Answer RQ1 directly in the first paragraph. Do not merely describe the table. Say whether ADP wins, by how much, and on which task families.

7.2 Where does ADP help most?

Break results down by:

- coding vs browsing vs mixed tasks,
- short vs long-horizon tasks,
- repeated subproblems vs novel tasks.

7.3 When does retrieval hurt?

This subsection is important for NeurIPS E&D positioning. Show:

- cases where all retrieval hurts,
- cases where ADP hurts less,
- cases where structure helps only when retrieval is selective.

Table 2: Ablation of structured components within ADP memory.

Variant	Success $\uparrow$	Usefulness $\uparrow$	Harm $\downarrow$	Cost $\downarrow$
Full ADP	–	–	–	–
w/o action types	–	–	–	–
w/o observations	–	–	–	–
w/o tool kwargs	–	–	–	–
w/o metadata	–	–	–	–
token-matched flat text	–	–	–	–

8 Ablations and Analysis

8.1 Which parts of ADP matter?

Ablate:

- remove action types,
- remove tool kwargs,
- remove observations,
- remove metadata,
- reorder or collapse steps,
- control for token count.

8.2 Representation vs retrieval-control

If you include control policies, this is where they belong. Make the interaction explicit: does ADP still help under always-retrieve? does its gain increase under selective search?

8.3 Noise and staleness stress tests

Corrupt the memory bank and measure robustness:

- irrelevant memories,
- stale trajectories,
- conflicting procedures,
- partial trajectories.

8.4 Qualitative case studies

Provide 2–4 short examples:

1. one success due to preserved procedure,
2. one failure due to misleading lexical similarity,

3. one failure of ADP itself,
4. one case where no memory was best.

9 Optional: Hierarchical Retrieval-Control Formulation

We optionally formalize retrieval-control as a hierarchical decision process. At step t , the wrapper observes a local state x_t and derives a compact query fingerprint F_t . A first-stage policy decides whether to query memory. If retrieval is triggered, the system obtains candidate memories

$$C_t = \{c_1, \dots, c_k\},$$

scores them, and then chooses one of the actions

$$\mathcal{A} = \{\text{NoSearch}, \text{SearchNoInject}, \text{InjectSummary}, \text{InjectFull}\}.$$

The reason for this decomposition is that some relevance quantities are only observable after retrieval. Thus, the first-stage policy is not a full utility estimator; it is only a search gate.

Recommendation. If this section does not have strong experimental support, keep only one paragraph in the main paper and move the equations to the appendix.

10 Discussion

10.1 What the results mean

Interpret the results at the representation level, not just the system level. Examples:

- If ADP wins mostly on procedural tasks, say that its value lies in preserving operational structure.
- If summary memory is competitive on easy tasks, say that structure is not always necessary.
- If ADP helps retrieval but not injection, separate the two bottlenecks explicitly.

10.2 Implications for future memory systems

Argue that trajectory representation is a first-class design choice in agent memory. Future work should not only optimize retrievers and rerankers, but also ask what kind of memory object is worth retrieving in the first place.

11 Limitations

This paper has several limitations. First, the current study may focus on one primary agent framework and one specific memory backend, which limits generalization across frameworks and storage systems. Second, the present RECALL implementation may represent each session as a single indexed item, which is coarser than segment-level or subtrajectory memory. Third, the evaluation may not fully disentangle representation quality from query construction quality. Fourth, any trigger or injection mechanism studied here should be understood as an initial policy rather than a definitive optimal controller. Finally, the tasks considered in this paper may emphasize procedural memory more than all real-world agent deployments do.

12 Reproducibility Statement

We will release the code, prompts, configuration files, conversion scripts, trajectory schemas, evaluation harness, and analysis scripts needed to reproduce the reported results. The released artifact will include: (i) conversion from raw session logs to ADP, (ii) memory construction for all compared representations, (iii) retrieval and evaluation scripts, (iv) exact task splits, (v) metric implementations, and (vi) instructions for reproducing the main tables and figures.

13 Conclusion

This paper asked whether ADP, originally proposed as a standardized format for offline agent training data, also serves as a superior memory representation for retrieval-augmented agents. By treating trajectory representation as an evaluative variable rather than a fixed implementation detail, we showed how structured procedural memory can be studied rigorously. Our results provide evidence about when structure helps, which components matter, and when retrieval is beneficial versus harmful. More broadly, the paper argues that representation choice is a central part of the science of agent memory, not merely an engineering convenience.

References

A Appendix Overview

This appendix provides implementation details, extended experiments, additional qualitative examples, and optional mathematical material omitted from the main paper for space.

B Implementation Details

B.1 Current Recall repository

List only the modules you truly have. For example:

- configuration,
- session-to-ADP conversion,
- embedding,
- vector-store ingestion,
- offline ingestion scripts.

B.2 Online evaluation harness

Describe any code added beyond the current repository for the experiments.

C Prompting, Models, and Runtime Configuration

Specify:

- base model(s),
- decoding settings,
- tool environment,
- retrieval top- k ,
- summary budgets,
- context-window budgets,
- seed management.

D Metric Definitions

Give exact formulas or pseudocode for:

- retrieval usefulness,
- injection harm,
- search cost,
- robustness metrics.

E Extended Tables

Put full per-task-family tables here.

F Additional Case Studies

Show longer examples of retrieved trajectories and injection outcomes.

G Optional Formalization of the Trigger

If retained, move the full trigger equations here.

G.1 Local state and fingerprint

$$x_t = (g_t, h_t, a_t, e_t, d_t, r_t), \quad F_t = \phi(x_t).$$

G.2 Candidate retrieval

$$C_t = \{c_1, \dots, c_k\}.$$

G.3 Candidate scoring

For each candidate c_i , define:

$$s_i \in [0, 1], \quad v_i \in [0, 1], \quad m_i \in [0, 1], \quad q_i \in [0, 1], \quad p_i \in [0, 1].$$

A generic ranking rule may be written as

$$R_i = \alpha s_i + \beta v_i + \gamma m_i + \delta q_i - \lambda p_i, \quad R_t^* = \max_i R_i.$$

Keep this generic unless you have experimental justification for fixed weights.

G.4 Search gate

If you keep the local gate:

$$L_t = \Gamma_d(t)(w_E E_t + w_N N_t^{\text{loc}} + w_U U_t),$$

with a hard-trigger override and a feasibility constraint for budget. Again, avoid overclaiming any fixed constants unless validated.

H Artifact Checklist for Release

- code repository,
- dependency file,
- scripts to regenerate memory banks,
- evaluation runner,
- seed list,

- anonymized sample data or instructions for reconstruction,
- README with exact commands,
- license statement.